

JavaScript Interview Questions & Answers

50+ Page Handbook for Internship, Fresher, and 10-15 LPA
Company Interviews

Theory + Code + Tricky Output + DOM + Async + Polyfills +
Project Questions

Document	JavaScript Interview PDF
Target	JavaScript / Frontend / MERN Stack Internships
Level	Beginner to Strong Fresher Interview Level
Best Use	15-20 days revision + mock interview practice

Goal: This PDF is designed to help you answer JavaScript questions confidently in interviews. Answers are written in simple interview English with Hinglish guidance where useful.

Table of Contents

1. How to Use This PDF
2. JavaScript Fundamentals
3. Variables, Scope, Hoisting, and TDZ
4. Functions, this, Closures, and Currying
5. Arrays and Objects
6. ES6+ Features
7. DOM and Events
8. Asynchronous JavaScript
9. Prototypes, Classes, and OOP
10. Browser, Storage, and Web APIs
11. Common Coding Questions with Solutions
12. Polyfills Asked in Interviews
13. Tricky Output Questions
14. Project and HR Based JavaScript Questions
15. 15-Day Revision Plan
16. Final Interview Cheat Sheets

Note: Page numbers can shift slightly depending on viewer, but the content is organized section-wise for fast revision.

1. How to Use This PDF

Do not read this like a novel. First understand the concept, then speak the answer aloud in your own words, then write the code once without looking. For interviews, confidence comes from active recall, not only reading.

For 10-15 LPA fresher or internship roles, companies usually test three things: JavaScript fundamentals, practical coding ability, and project explanation. This PDF covers all three in one place.

Daily method: 10 theory questions + 5 output questions + 2 coding questions. Speak each answer in 60-90 seconds as if the interviewer is in front of you.

2. JavaScript Fundamentals

Q1. What is JavaScript?

JavaScript is a high-level, interpreted programming language mainly used to make web pages interactive. It runs in browsers and also on servers using Node.js. In interviews, say that JavaScript is single-threaded, dynamically typed, prototype-based, and supports asynchronous programming.

Q2. Is JavaScript compiled or interpreted?

Traditionally JavaScript is called interpreted, but modern engines like V8 use Just-In-Time compilation. So a strong answer is: JavaScript source code is parsed, optimized, and compiled internally by the engine during execution.

Q3. What are primitive data types in JavaScript?

Primitive types are string, number, bigint, boolean, undefined, symbol, and null. They are immutable and compared by value. Objects, arrays, and functions are non-primitive and compared by reference.

Q4. What is the difference between null and undefined?

undefined means a variable is declared but value is not assigned, or a function does not return anything. null is an intentional empty value assigned by the developer.

Q5. What is dynamic typing?

Dynamic typing means the variable type is decided at runtime. A variable can hold a number first and later hold a string. This gives flexibility but can also create bugs if types are not handled carefully.

Q6. What is type coercion?

Type coercion is automatic conversion of one data type into another during operations. For example, '5' + 2 becomes '52' because + with string performs concatenation, but '5' - 2 becomes 3 because subtraction expects numbers.

Q7. Difference between == and ===?

== checks equality after type conversion. === checks equality without type conversion. In interviews, always say prefer === because it avoids unexpected coercion bugs.

Q8. What is NaN?

NaN means Not-a-Number. It is a special number value that represents an invalid numeric operation. Interesting point: NaN is not equal to itself, so use Number.isNaN(value) to check it.

Q9. What is truthy and falsy?

Falsy values are false, 0, -0, 0n, empty string, null, undefined, and NaN. Everything else is truthy, including empty arrays and empty objects.

Q10. What are template literals?

Template literals use backticks and allow string interpolation using `${}`. They also support multiline strings, making code cleaner than string concatenation.

Q11. What is short-circuit evaluation?

In logical operators, JavaScript may stop evaluating once the final result is known. In `a || b`, if `a` is truthy, `b` is not checked. In `a && b`, if `a` is falsy, `b` is not checked.

Q12. What is the difference between expression and statement?

An expression produces a value, like `2 + 3` or a function call. A statement performs an action, like `if`, `for`, or `return`. This matters when understanding arrow functions and JSX.

Q13. What is strict mode?

Strict mode is a safer JavaScript mode enabled using `'use strict'`. It catches common mistakes, prevents accidental globals, and changes some silent errors into real errors.

Q14. What is immutability?

Immutability means not changing existing data directly. Instead, create a new copy with updated values. This is very important in React state updates and predictable programming.

Q15. Why is JavaScript single-threaded?

JavaScript has one main call stack, so it executes one piece of synchronous code at a time. But browser APIs, the callback queue, microtask queue, and event loop help JavaScript handle asynchronous tasks.

```
console.log(typeof "Alex");      // string
console.log(typeof 10);          // number
console.log(typeof null);        // object (historical bug)
console.log(Number.isNaN(NaN));  // true
console.log([] == false);        // true because coercion happens
console.log([] === false);       // false because types differ
```

3. Variables, Scope, Hoisting, and TDZ

Q16. Difference between var, let, and const?

var is function-scoped and can be redeclared. let and const are block-scoped. let can be reassigned, const cannot be reassigned. const does not mean object is fully immutable; object properties can still be changed.

Q17. What is scope?

Scope defines where a variable can be accessed. JavaScript has global scope, function scope, and block scope. let and const follow block scope; var follows function scope.

Q18. What is lexical scope?

Lexical scope means scope is decided by where the function is written in the code, not where it is called. Inner functions can access variables from outer functions.

Q19. What is hoisting?

Hoisting is JavaScript behavior where declarations are moved to the top of their scope during the creation phase. var is hoisted with undefined; let and const are hoisted but stay in the Temporal Dead Zone.

Q20. What is Temporal Dead Zone?

TDZ is the time between entering a scope and the actual let/const declaration line. Accessing the variable before declaration gives ReferenceError.

Q21. Can const object properties be changed?

Yes. const prevents reassignment of the variable reference, not mutation of the object. To prevent mutation, use Object.freeze for shallow freezing.

Q22. What is shadowing?

Shadowing happens when an inner scope variable has the same name as an outer scope variable. The inner variable hides the outer one inside that block or function.

Q23. What is illegal shadowing?

In JavaScript, declaring var inside a block to shadow a let variable from outer scope can create syntax issues in some cases. This is because var is function-scoped and conflicts with block-scoped declaration.

Q24. What is global object in browser?

In browsers, the global object is window. Global var declarations become properties of window, but let and const do not become window properties.

Q25. What is the scope chain?

When JavaScript cannot find a variable in the current scope, it looks in the outer lexical scope, then further outer scopes, until global scope. This lookup path is called the scope chain.

```
console.log(a); // undefined because var is hoisted
var a = 10;
```

```
// console.log(b); // ReferenceError because b is in TDZ
let b = 20;
```

```
const user = { name: "Alex" };
user.name = "Doe" ; // allowed
// user = {};      // not allowed
```


4. Functions, this, Closures, and Currying

Q26. What are functions in JavaScript?

Functions are reusable blocks of code. In JavaScript, functions are first-class citizens, which means they can be stored in variables, passed as arguments, and returned from other functions.

Q27. Function declaration vs function expression?

Function declarations are hoisted completely, so they can be called before definition. Function expressions are assigned to variables, so they follow variable hoisting rules.

Q28. What is an arrow function?

Arrow functions provide shorter syntax and do not have their own `this`, `arguments`, `super`, or `new.target`. They take `this` from the surrounding lexical scope.

Q29. What is callback function?

A callback is a function passed into another function to be executed later. It is common in event handling, array methods, and asynchronous operations.

Q30. What is higher-order function?

A higher-order function either takes another function as an argument or returns a function. `map`, `filter`, and `reduce` are common examples.

Q31. What is closure?

A closure is created when an inner function remembers variables from its outer function even after the outer function has finished execution.

Closures are used in data privacy, function factories, `debounce`, `throttle`, and memoization.

Q32. Why are closures useful?

Closures help keep data private, create reusable functions, and maintain state without using global variables. Interviewers often ask closure output questions.

Q33. What is this keyword?

this refers to the execution context. Its value depends on how a function is called: object method, normal function, constructor, arrow function, or explicit binding using call/apply/bind.

Q34. Difference between call, apply, and bind?

call invokes a function immediately with arguments individually. apply invokes immediately with arguments as an array. bind returns a new function with fixed this and optional preset arguments.

Q35. What is currying?

Currying transforms a function with multiple arguments into a sequence of functions, each taking one argument. It is useful for reusable and partially applied functions.

Q36. What is recursion?

Recursion is when a function calls itself until it reaches a base case. It is useful for tree traversal, factorial, Fibonacci, nested data, and backtracking.

Q37. What is IIFE?

IIFE means Immediately Invoked Function Expression. It runs immediately after creation and was commonly used to create private scope before ES6 modules and let/const.

Q38. What is memoization?

Memoization is an optimization technique where expensive function results are stored in a cache. If the same input comes again, return from cache instead of recalculating.

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    return count;
  };
}

const counter = outer();
console.log(counter()); // 1
console.log(counter()); // 2

const user = {
  name: "Alex",
  normal() { return this.name; },
  arrow: () => this.name
}
```

```

};
console.log(user.normal()); // Alex
console.log(user.arrow()); // undefined in most browser/module cases

function sum(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    };
  };
}
console.log(sum(1)(2)(3)); // 6

function memoizedSquare() {
  const cache = {};
  return function(n) {
    if (cache[n]) return cache[n];
    console.log("Calculating...");
    cache[n] = n * n;
    return cache[n];
  };
}
const square = memoizedSquare();
console.log(square(5));
console.log(square(5)); // cache used

```

5. Arrays and Objects

Q39. Difference between map, filter, and reduce?

map transforms every element and returns a new array. filter selects elements based on a condition. reduce converts the array into a single value like sum, object, or grouped result.

Q40. Difference between forEach and map?

forEach is used for side effects and returns undefined. map returns a new array, so it is used when transformation is needed.

Q41. What is destructuring?

Destructuring extracts values from arrays or properties from objects into variables in a clean way.

Q42. What is spread operator?

Spread expands arrays or objects. It is commonly used to copy arrays, merge arrays, copy objects, and pass array values as function arguments.

Q43. What is rest operator?

Rest collects remaining values into an array. It is used in function parameters and destructuring.

Q44. Difference between shallow copy and deep copy?

Shallow copy copies only the top level. Nested objects still share references. Deep copy creates independent copies of nested objects as well.

Q45. How to clone an object?

For shallow clone use spread operator or Object.assign. For deep clone, structuredClone is preferred for many modern cases, or JSON.parse(JSON.stringify(obj)) with limitations.

Q46. What is optional chaining?

Optional chaining ?. safely accesses nested properties without throwing error if a property is null or undefined.

Q47. What is nullish coalescing?

?? returns the right value only when the left side is null or undefined. It is better than || when 0 or empty string are valid values.

Q48. Difference between Object.keys, Object.values, Object.entries?

Object.keys returns property names, Object.values returns values, and Object.entries returns key-value pairs as arrays.

Q49. What is array destructuring swap?

You can swap variables using array destructuring without a temporary variable: [a, b] = [b, a].

Q50. What are Set and Map?

Set stores unique values. Map stores key-value pairs and allows any type as key. Map is better when keys are not only strings or when insertion order matters.

```
const nums = [1, 2, 3, 4];
console.log(nums.map(n => n * 2));           // [2, 4, 6, 8]
console.log(nums.filter(n => n % 2));        // [1, 3]
console.log(nums.reduce((sum, n) => sum + n, 0)); // 10

const user = { name: "Alex", address: { city: "City" } };
const shallow = { ...user };
shallow.address.city = "Delhi";
console.log(user.address.city); // Delhi, because nested object shared

const deep = structuredClone(user);
deep.address.city = "Noida";
console.log(user.address.city); // Delhi
```

6. ES6+ Features

Q51. What important features came in ES6?

ES6 introduced let/const, arrow functions, template literals, destructuring, default parameters, rest/spread, classes, promises, modules, Map, Set, and enhanced object literals.

Q52. What are default parameters?

Default parameters allow a function to use a default value when an argument is missing or undefined.

Q53. What are modules?

Modules allow splitting code into reusable files using export and import. They help maintain clean architecture in larger applications.

Q54. Named export vs default export?

Named exports can export multiple values and must be imported using the exact exported names unless aliased. Default export exports one main value from a file and can be imported with any name.

Q55. What are enhanced object literals?

Enhanced object literals allow shorthand properties, shorthand methods, and computed property names inside objects.

Q56. What is Symbol?

Symbol is a primitive type used to create unique identifiers. Even two symbols with the same description are different.

Q57. What is BigInt?

BigInt represents integers larger than `Number.MAX_SAFE_INTEGER`. It is written with `n` suffix, like `9007199254740993n`.

Q58. What are iterables and iterators?

An iterable is an object that can be looped using `for...of`. It must implement `Symbol.iterator`. Arrays, strings, maps, and sets are iterable.

Q59. What is generator function?

A generator function uses `function*` and `yield` to pause and resume execution. It returns an iterator.

Q60. What is optional chaining useful for in React/Node?

It helps safely access API response data where nested values may be missing. Example: `user?.profile?.name` avoids runtime crashes.

```
// utils.js
export const add = (a, b) => a + b;
export default function greet(name) {
  return `Hello ${name}`;
}

// app.js
import greet, { add } from "./utils.js";
console.log(greet("Alex"));
console.log(add(2, 3));

function* idGenerator() {
  yield 1;
  yield 2;
  yield 3;
}
for (const id of idGenerator()) console.log(id);
```

7. DOM and Events

Q61. What is DOM?

DOM stands for Document Object Model. It represents HTML as a tree of objects so JavaScript can read and manipulate page content, styles, and structure.

Q62. `querySelector` vs `getElementById`?

`getElementById` selects one element by id and is fast. `querySelector` selects the first matching CSS selector and is more flexible.

Q63. `innerHTML` vs `textContent`?

`innerHTML` parses HTML and can create elements, but can be risky with untrusted input. `textContent` treats content as plain text and is safer.

Q64. What is event bubbling?

Event bubbling means an event starts from the target element and moves upward to parent elements. This is the default phase used in most event listeners.

Q65. What is event capturing?

Event capturing means the event travels from the root down to the target element. It can be enabled by passing `true` or `{ capture: true }` in `addEventListener`.

Q66. What is event delegation?

Event delegation means adding one event listener to a parent and handling child events using `event.target`. It improves performance and works for dynamically added elements.

Q67. `preventDefault` vs `stopPropagation`?

`preventDefault` stops the default browser behavior. `stopPropagation` stops the event from moving further in bubbling or capturing.

Q68. What is dataset?

`dataset` allows reading custom `data-*` attributes from HTML elements in JavaScript.

Q69. What is DOMContentLoaded?

DOMContentLoaded fires when HTML is fully parsed, without waiting for images and stylesheets. load event waits for all resources.

Q70. How do you create an element dynamically?

Use document.createElement, set text or attributes, then append it using appendChild or append.

```
<ul id="taskList">
  <li data-id="1">Learn JS</li>
  <li data-id="2">Practice DOM</li>
</ul>

<script>
const list = document.querySelector("#taskList");
list.addEventListener("click", function(e) {
  if (e.target.tagName === "LI") {
    console.log("Clicked task id:", e.target.dataset.id);
  }
});
</script>
```

8. Asynchronous JavaScript

Q71. What is asynchronous JavaScript?

Asynchronous JavaScript allows long tasks like API calls, timers, and file operations to run without blocking the main thread. It improves user experience.

Q72. What is callback hell?

Callback hell happens when callbacks are nested too deeply, making code hard to read and maintain. Promises and `async/await` solve this problem.

Q73. What is Promise?

A Promise is an object representing the eventual success or failure of an asynchronous operation. It has pending, fulfilled, and rejected states.

Q74. `then/catch/finally`?

`then` handles success, `catch` handles errors, and `finally` runs after promise settles whether fulfilled or rejected.

Q75. What is `async/await`?

`async/await` is syntax over promises that makes asynchronous code look like synchronous code. `await` pauses inside `async` function until the promise settles.

Q76. What is event loop?

The event loop coordinates the call stack, web APIs, microtask queue, and callback queue. It pushes tasks to the stack when the stack is empty.

Q77. Microtask vs macrotask?

Promise callbacks and `queueMicrotask` go into microtask queue. `setTimeout` and `setInterval` callbacks go into macrotask queue. Microtasks run before the next macrotask.

Q78. `Promise.all` vs `Promise.allSettled`?

`Promise.all` fails fast if any promise rejects. `Promise.allSettled` waits for all promises and returns each result status.

Q79. Promise.race vs Promise.any?

Promise.race settles with the first promise that settles, success or failure. Promise.any resolves with the first fulfilled promise and ignores rejections unless all fail.

Q80. How to handle errors in async/await?

Use try/catch around await calls. For multiple requests, decide whether you need Promise.all, allSettled, or sequential awaits.

Q81. What is fetch?

fetch is a browser API for making HTTP requests. It returns a Promise. Important: fetch rejects mainly on network errors, not on HTTP 404/500, so check response.ok.

Q82. What is AbortController?

AbortController can cancel fetch requests. It is useful in search boxes, React effects, and avoiding outdated API responses.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");

// Output: A D C B
// Reason: sync code first, then microtasks, then macrotasks.

async function getUser() {
  try {
    const res = await fetch("https://api.example.com/user");
    if (!res.ok) throw new Error("HTTP Error: " + res.status);
    const data = await res.json();
    return data;
  } catch (error) {
    console.error("Failed:", error.message);
  }
}
```

9. Prototypes, Classes, and OOP

Q83. What is prototype in JavaScript?

Every JavaScript object has an internal link to another object called its prototype. If a property is not found on the object, JavaScript searches the prototype chain.

Q84. What is prototype chain?

The prototype chain is the lookup path JavaScript follows from an object to its prototype, then prototype's prototype, until null.

Q85. What is constructor function?

A constructor function is used with new to create objects. It sets this to the new object and can initialize properties.

Q86. What happens when we use new?

new creates an empty object, links it to the constructor's prototype, binds this to the new object, executes the constructor, and returns the object unless a non-primitive object is returned explicitly.

Q87. Are classes real classes in JavaScript?

JavaScript classes are syntactic sugar over prototypes. They provide cleaner syntax but internally use prototype-based inheritance.

Q88. What is inheritance?

Inheritance allows one class or object to use properties and methods from another. In JavaScript, it is based on prototypes.

Q89. What is super?

super is used inside child classes to call parent constructor or parent methods.

Q90. What is encapsulation?

Encapsulation means hiding internal data and exposing only controlled methods. In JavaScript, closures and private class fields can help with encapsulation.

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.sayHi = function() {
```

```
    return "Hi " + this.name;
};

const p1 = new Person("Alex");
console.log(p1.sayHi());

class Developer extends Person {
  constructor(name, skill) {
    super(name);
    this.skill = skill;
  }
  intro() {
    return `${this.name} knows ${this.skill}`;
  }
}

const dev = new Developer("Alex", "MERN");
console.log(dev.intro());
```

10. Browser, Storage, and Web APIs

Q91. localStorage vs sessionStorage?

localStorage persists until manually cleared. sessionStorage persists only for the current tab session. Both store string key-value pairs.

Q92. What are cookies?

Cookies are small pieces of data stored by the browser and sent with HTTP requests to matching domains. They are often used for sessions and authentication.

Q93. localStorage vs cookies?

localStorage is not sent automatically with requests and has more storage. Cookies are sent with requests and can be configured with HttpOnly, Secure, SameSite for auth security.

Q94. What is CORS?

CORS means Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether frontend from one origin can access resources from another origin.

Q95. What is same-origin policy?

Same-origin policy restricts how documents or scripts from one origin interact with resources from another origin. Origin includes protocol, domain, and port.

Q96. What is debouncing in frontend?

Debouncing delays function execution until the user stops triggering it for a specified time. It is common in search input to reduce API calls.

Q97. What is throttling in frontend?

Throttling ensures a function runs at most once in a specified time interval. It is common in scroll and resize events.

Q98. What is memory leak in frontend?

A memory leak happens when memory is no longer needed but still referenced. Common causes are unremoved event listeners, uncleared intervals, and retained DOM references.

Q99. What is reflow and repaint?

Reflow recalculates layout; repaint redraws pixels. Frequent DOM style changes can cause performance issues.

Q100. How to improve JS performance?

Use event delegation, avoid unnecessary DOM access, debounce expensive events, use efficient data structures, lazy load resources, and avoid blocking the main thread.

```
localStorage.setItem("token", "abc123");
console.log(localStorage.getItem("token"));
localStorage.removeItem("token");

// Debounce example used in search box
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

11. Common Coding Questions with Solutions

Q101. Reverse a string

Split the string into characters, reverse the array, then join it again.

```
function reverseString(str) {  
  return str.split('').reverse().join('');  
}  
console.log(reverseString('javascript')); // tpircsavaj
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q102. Check palindrome

Compare the string with its reversed version after normalizing case.

```
function isPalindrome(str) {  
  const s = str.toLowerCase().replace(/[^\a-z0-9]/g, '');  
  return s === s.split('').reverse().join('');  
}  
console.log(isPalindrome('Madam')); // true
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q103. Find max in array

Keep one variable max and update it while traversing.

```
function findMax(arr) {  
  let max = -Infinity;  
  for (const num of arr) {  
    if (num > max) max = num;  
  }  
  return max;  
}  
console.log(findMax([3, 9, 2, 11])); // 11
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q104. Remove duplicates

Use Set because Set stores unique values.

```
function removeDuplicates(arr) {  
  return [...new Set(arr)];  
}  
console.log(removeDuplicates([1,2,2,3,3,4])); // [1,2,3,4]
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q105. Count frequency

Use an object or Map to count how many times each value appears.

```
function frequency(arr) {
  const map = {};
  for (const item of arr) {
    map[item] = (map[item] || 0) + 1;
  }
  return map;
}
console.log(frequency(['a','b','a'])); // { a: 2, b: 1 }
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q106. Find first non-repeating character

Count characters, then traverse again to find first character with frequency 1.

```
function firstUnique(str) {
  const freq = {};
  for (const ch of str) freq[ch] = (freq[ch] || 0) + 1;
  for (const ch of str) if (freq[ch] === 1) return ch;
  return null;
}
console.log(firstUnique('aabbcd')); // c
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q107. Factorial using recursion

Base case is $n \leq 1$. Recursive case is $n * \text{factorial}(n - 1)$.

```
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
console.log(factorial(5)); // 120
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q108. Fibonacci using iteration

Use two variables to avoid exponential recursion.

```
function fibonacci(n) {
  if (n <= 1) return n;
  let a = 0, b = 1;
  for (let i = 2; i <= n; i++) {
    const c = a + b;
    a = b;
    b = c;
  }
}
```

```
    return b;
  }
  console.log(fibonacci(6)); // 8
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q109. Flatten an array

Use recursion. If item is array, flatten it; otherwise push item.

```
function flatten(arr) {
  const result = [];
  for (const item of arr) {
    if (Array.isArray(item)) result.push(...flatten(item));
    else result.push(item);
  }
  return result;
}
console.log(flatten([1,[2,[3,4]],5])); // [1,2,3,4,5]
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q110. Deep clone object

Use recursion to clone arrays and objects. Handle primitives directly.

```
function deepClone(value) {
  if (value === null || typeof value !== 'object') return value;
  if (Array.isArray(value)) return value.map(deepClone);
  const clone = {};
  for (const key in value) clone[key] = deepClone(value[key]);
  return clone;
}
const obj = { a: 1, b: { c: 2 } };
const copy = deepClone(obj);
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q111. Debounce function

Clear the previous timer and start a new timer each time the returned function is called.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
const search = debounce((q) => console.log(q), 500);
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q112. Throttle function

Run the function only if enough time has passed since the last call.

```
function throttle(fn, limit) {  
  let lastCall = 0;  
  return function(...args) {  
    const now = Date.now();  
    if (now - lastCall >= limit) {  
      lastCall = now;  
      fn.apply(this, args);  
    }  
  };  
}
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q113. Check anagram

Sort both strings or count frequency. Frequency approach is better.

```
function isAnagram(a, b) {  
  if (a.length !== b.length) return false;  
  const freq = {};  
  for (const ch of a) freq[ch] = (freq[ch] || 0) + 1;  
  for (const ch of b) {  
    if (!freq[ch]) return false;  
    freq[ch]--;  
  }  
  return true;  
}  
console.log(isAnagram('listen', 'silent')); // true
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q114. Two sum

Use a Map to store visited numbers and their indices.

```
function twoSum(nums, target) {  
  const map = new Map();  
  for (let i = 0; i < nums.length; i++) {  
    const need = target - nums[i];  
    if (map.has(need)) return [map.get(need), i];  
    map.set(nums[i], i);  
  }  
  return [];  
}  
console.log(twoSum([2,7,11,15], 9)); // [0,1]
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q115. Merge two sorted arrays

Use two pointers and push the smaller element.

```
function mergeSorted(a, b) {
  let i = 0, j = 0, res = [];
  while (i < a.length && j < b.length) {
    if (a[i] <= b[j]) res.push(a[i++]);
    else res.push(b[j++]);
  }
  return res.concat(a.slice(i)).concat(b.slice(j));
}
console.log(mergeSorted([1,3,5],[2,4,6]));
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q116. Implement once function

Run a function only once and cache its result.

```
function once(fn) {
  let called = false;
  let result;
  return function(...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);
    }
    return result;
  };
}
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q117. Group by property

Use reduce to create an object where each key contains matching items.

```
function groupBy(arr, key) {
  return arr.reduce((acc, item) => {
    const group = item[key];
    if (!acc[group]) acc[group] = [];
    acc[group].push(item);
    return acc;
  }, {});
}
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q118. Capitalize each word

Split by space, capitalize first char, then join.

```
function titleCase(str) {  
  return str.split(' ').map(word =>  
    word.charAt(0).toUpperCase() + word.slice(1).toLowerCase()  
  ).join(' ');  
}  
console.log(titleCase('hello JavaScript Interview PDF'));
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q119. Find missing number

Use sum formula or XOR. Sum formula is simple for 0 to n.

```
function missingNumber(nums) {  
  const n = nums.length;  
  const expected = n * (n + 1) / 2;  
  const actual = nums.reduce((s, x) => s + x, 0);  
  return expected - actual;  
}  
console.log(missingNumber([3,0,1])); // 2
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

Q120. Valid parentheses

Use stack. Push opening brackets, pop and match when closing bracket appears.

```
function isValid(s) {  
  const stack = [];  
  const pairs = { ')': '(', '}': '{', ']': '[' };  
  for (const ch of s) {  
    if (ch === '(' || ch === '{' || ch === '[') stack.push(ch);  
    else if (stack.pop() !== pairs[ch]) return false;  
  }  
  return stack.length === 0;  
}  
console.log(isValid('{[]}')); // true
```

Interview tip: First explain approach, then write code. Interviewer wants thinking clarity more than memorized syntax.

12. Polyfills Asked in Interviews

Q121. Polyfill for map

Implement map by looping over array and pushing callback result into a new array.

```
Array.prototype.myMap = function(callback) {  
  const result = [];  
  for (let i = 0; i < this.length; i++) {  
    if (i in this) result.push(callback(this[i], i, this));  
  }  
  return result;  
};  
console.log([1,2,3].myMap(x => x * 2));
```

Interview tip: Polyfills are asked to check your understanding of internals, callback, this, and async behavior.

Q122. Polyfill for filter

Push only those elements for which callback returns true.

```
Array.prototype.myFilter = function(callback) {  
  const result = [];  
  for (let i = 0; i < this.length; i++) {  
    if (i in this && callback(this[i], i, this)) result.push(this[i]);  
  }  
  return result;  
};
```

Interview tip: Polyfills are asked to check your understanding of internals, callback, this, and async behavior.

Q123. Polyfill for reduce

Maintain accumulator. If initial value is not passed, start from first element.

```
Array.prototype.myReduce = function(callback, initialValue) {  
  let acc = initialValue;  
  let start = 0;  
  if (acc === undefined) {  
    acc = this[0];  
    start = 1;  
  }  
  for (let i = start; i < this.length; i++) {  
    acc = callback(acc, this[i], i, this);  
  }  
  return acc;  
};
```

Interview tip: Polyfills are asked to check your understanding of internals, callback, this, and async behavior.

Q124. Polyfill for bind

Return a new function that calls original function with fixed this and combined arguments.

```
Function.prototype.myBind = function(context, ...args1) {  
  const fn = this;  
  return function(...args2) {  
    return fn.apply(context, [...args1, ...args2]);  
  };  
};  
function greet(city) { return this.name + ' from ' + city; }  
const user = { name: 'Alex' };  
console.log(greet.myBind(user)('City'));
```

Interview tip: Polyfills are asked to check your understanding of internals, callback, this, and async behavior.

Q125. Polyfill for Promise.all

Run all promises, store results by index, resolve when all complete, reject immediately if any fails.

```
function myPromiseAll(promises) {  
  return new Promise((resolve, reject) => {  
    const results = [];  
    let completed = 0;  
    if (promises.length === 0) resolve(results);  
    promises.forEach((promise, index) => {  
      Promise.resolve(promise).then(value => {  
        results[index] = value;  
        completed++;  
        if (completed === promises.length) resolve(results);  
      }).catch(reject);  
    });  
  });  
}
```

Interview tip: Polyfills are asked to check your understanding of internals, callback, this, and async behavior.

13. Tricky Output Questions

Q126. Output?

var is function-scoped, so loop completes before callbacks run. All callbacks print 3.

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 100);  
}  
// Output: 3 3 3
```

Q127. Output?

let is block-scoped, so each iteration has a new i.

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 100);  
}  
// Output: 0 1 2
```

Q128. Output?

Object keys are strings except symbols. Here true becomes 'true'.

```
const obj = {};  
obj[true] = 'yes';  
console.log(obj['true']);  
// Output: yes
```

Q129. Output?

Arrays are objects, so comparing two different array references gives false.

```
console.log([] == []);  
console.log([] === []);  
// Output: false false
```

Q130. Output?

+ operator with array converts array to string.

```
console.log([1,2] + [3,4]);  
// Output: 1,23,4
```

Q131. Output?

Function declaration is hoisted completely.

```
sayHi();  
function sayHi() {  
  console.log('Hi');  
}  
// Output: Hi
```


Q132. Output?

Function expression follows var hoisting. greet is undefined at call time.

```
greet();  
var greet = function() { console.log('Hello'); };  
// Output: TypeError: greet is not a function
```

Q133. Output?

Promise microtask runs before setTimeout macrotask.

```
setTimeout(() => console.log('timeout'), 0);  
Promise.resolve().then(() => console.log('promise'));  
console.log('sync');  
// Output: sync promise timeout
```

Q134. Output?

this inside regular method points to object.

```
const user = { name: 'Alex', getName() { return this.name; } };  
console.log(user.getName());  
// Output: Alex
```

Q135. Output?

Arrow function does not have own this, so it does not point to user object.

```
const user = { name: 'Alex', getName: () => this.name };  
console.log(user.getName());  
// Output: undefined in most cases
```

Q136. Output?

Object property access checks own property first, then prototype.

```
const parent = { city: 'Delhi' };  
const child = Object.create(parent);  
child.name = 'Alex';  
console.log(child.city);  
// Output: Delhi
```

Q137. Output?

typeof null is object due to old JavaScript bug.

```
console.log(typeof null);  
// Output: object
```

Q138. Output?

NaN is not equal to itself.

```
console.log(NaN === NaN);  
console.log(Number.isNaN(NaN));  
// Output: false true
```

Q139. Output?

0 is falsy, but [] is truthy.

```
if ([]) console.log('array truthy');
if (0) console.log('zero truthy');
// Output: array truthy
```

Q140. Output?

Closure remembers outer variable count.

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const c = makeCounter();
console.log(c());
console.log(c());
// Output: 1 2
```

Q141. Output?

const prevents reassignment, not object mutation.

```
const obj = { a: 1 };
obj.a = 2;
console.log(obj.a);
// Output: 2
```

Q142. Output?

delete removes object property, not normal local variable.

```
const obj = { a: 1 };
delete obj.a;
console.log(obj.a);
// Output: undefined
```

Q143. Output?

Spread makes shallow copy, nested object still shared.

```
const a = { x: { y: 1 } };
const b = { ...a };
b.x.y = 99;
console.log(a.x.y);
// Output: 99
```

Q144. Output?

|| returns first truthy value; && returns first falsy or last truthy value.

```
console.log('' || 'hello');
console.log('hi' && 10);
// Output: hello 10
```

Q145. Output?

?? only checks null or undefined, so 0 is kept.

```
console.log(0 || 10);  
console.log(0 ?? 10);  
// Output: 10 0
```

14. Project and HR Based JavaScript Questions

Q146. Where did you use JavaScript in your project?

Answer with project-specific confidence: I used JavaScript for UI logic, event handling, API integration, form validation, state updates, and handling asynchronous operations like fetch calls. In React/Node projects, I used JavaScript across frontend components and backend APIs.

Q147. Explain one difficult JavaScript bug you solved.

Use STAR format: Situation, Task, Action, Result. Example: API data was coming late and UI was breaking. I used loading states, optional chaining, and error handling to solve it.

Q148. How did you handle API errors?

I checked response status, used try/catch, displayed user-friendly error messages, and avoided app crash. For backend, I returned proper status codes and JSON responses.

Q149. How did you optimize frontend performance?

Mention debouncing search input, avoiding unnecessary renders, using event delegation, lazy loading, and minimizing direct DOM manipulation.

Q150. How do you explain async/await in your project?

Say: I used async/await to call APIs in readable form. I wrapped API calls in try/catch and updated UI based on loading, success, and error states.

Q151. How do you validate forms?

I validate required fields, email formats, password rules, and show clear error messages. On backend, I validate again because frontend validation alone is not secure.

Q152. How do you secure tokens?

For strong security, HttpOnly cookies are safer than localStorage for auth tokens because JavaScript cannot access HttpOnly cookies. For basic projects, localStorage is common but less secure against XSS.

Q153. What did you learn from JavaScript projects?

I learned that real development is not only syntax. It includes debugging, API handling, edge cases, performance, clean code, and user experience.

Q154. Why should we hire you for JavaScript internship?

I have built practical projects, understand JavaScript fundamentals, and I am actively improving DSA and full-stack skills. I can learn quickly, debug patiently, and contribute to real tasks.

Q155. What will you do if you do not know an answer?

Say honestly: I will first clarify the requirement, break the problem into smaller parts, check documentation if allowed, and try to reason from fundamentals instead of guessing blindly.

15. 15-Day JavaScript Interview Revision Plan

Day	Focus
Day 1	Data types, coercion, == vs ===, truthy/falsy, strict mode
Day 2	var/let/const, scope, hoisting, TDZ, shadowing
Day 3	Functions, callbacks, higher-order functions, arrow functions
Day 4	this keyword, call, apply, bind
Day 5	Closures, currying, IIFE, memoization
Day 6	Arrays: map/filter/reduce, sorting, destructuring
Day 7	Objects: reference, shallow/deep copy, Map, Set
Day 8	DOM selection, manipulation, events
Day 9	Event bubbling, capturing, delegation, forms
Day 10	Promises, async/await, fetch, error handling
Day 11	Event loop, microtask, macrotask, output questions
Day 12	Prototype, prototype chain, classes, inheritance
Day 13	Coding: debounce, throttle, flatten, deep clone
Day 14	Polyfills: map, filter, reduce, bind, Promise.all
Day 15	Mock interview: project explanation + 30 rapid questions

Mock routine: Record yourself answering 10 questions daily. Your goal is not perfect English; your goal is clear explanation. Slowly your fluency and confidence will improve.

16. Final Interview Cheat Sheets

var vs let vs const: var is function-scoped, let/const are block-scoped. Prefer const by default, let when reassignment is needed, avoid var in modern code.

Closure one-line: A closure is when an inner function remembers variables from its outer lexical scope even after the outer function has returned.

Event loop one-line: Synchronous code runs first, then microtasks like promises, then macrotasks like setTimeout.

this one-line: this depends on how the function is called; arrow functions take this from lexical scope.

Promise one-line: A Promise represents a future async result with pending, fulfilled, or rejected state.

Prototype one-line: Objects inherit properties and methods through prototype chain.

Debounce one-line: Debounce waits until events stop for a delay, useful for search inputs.

Throttle one-line: Throttle limits execution to once per interval, useful for scroll or resize.

Shallow copy one-line: Shallow copy copies top-level only; nested references are shared.

Deep copy one-line: Deep copy creates independent nested copies; structuredClone is useful in modern JavaScript.

Final advice: In interview, do not panic. First say your approach, then write code, then dry-run with one example. If stuck, explain your thought process. That shows maturity.

End of Handbook - Keep practicing. Consistency beats fear.

17. Extra Advanced JavaScript Practice Questions

Q156. What is execution context?

Execution context is the environment where JavaScript code runs. It has a memory phase and an execution phase. Global execution context is created first, and each function call creates a new function execution context.

Q157. What is call stack?

Call stack stores currently executing function calls. When a function is called, it is pushed to the stack. When it returns, it is popped from the stack. Stack overflow happens when too many calls are stacked, often due to missing recursion base case.

Q158. What is garbage collection?

Garbage collection automatically frees memory that is no longer reachable. JavaScript engines usually use mark-and-sweep algorithm to identify unreachable objects.

Q159. What is memory leak with event listener?

If an element is removed from DOM but event listener still references it, memory may not be released. Remove listeners when they are no longer needed, especially in single-page apps.

Q160. What is pure function?

A pure function returns the same output for the same input and has no side effects. Pure functions are easier to test, debug, and reuse.

Q161. What is side effect?

A side effect is any operation that changes something outside the function scope, like modifying global variables, changing DOM, writing to storage, or making API calls.

Q162. What is function composition?

Function composition means combining small functions to create a bigger function. Output of one function becomes input of another.

Q163. What is optional chaining output?

If any part before `?.` is null or undefined, result becomes undefined instead of throwing an error.

Q164. What is object freezing?

`Object.freeze` prevents adding, removing, or changing top-level properties. It is shallow, so nested objects can still be changed unless frozen separately.

Q165. What is `Object.seal`?

`Object.seal` prevents adding or deleting properties, but existing writable properties can still be changed.

Q166. What is `Object.create`?

`Object.create` creates a new object with a specified prototype. It is often used to understand prototypal inheritance.

Q167. What is event target vs `currentTarget`?

`event.target` is the actual clicked element. `event.currentTarget` is the element on which the event listener is attached.

Q168. What is passive event listener?

A passive listener tells the browser that the event handler will not call `preventDefault`. It can improve scroll performance.

Q169. What is `async defer` in script tag?

`async` downloads script without blocking and executes when ready. `defer` downloads without blocking but executes after HTML parsing in order.

Q170. What is module script?

A script with `type='module'` supports `import/export`, runs in strict mode by default, and has deferred behavior.

Q171. What is `JSON.stringify` limitation?

`JSON.stringify` cannot properly copy functions, undefined, Symbol, circular references, Date objects as Date, Map, Set, and some special values.

Q172. What is `structuredClone`?

`structuredClone` creates a deep clone of many built-in types and handles circular references, but it cannot clone functions or DOM nodes.

Q173. What is Promise chaining?

Promise chaining means returning values or promises from then callbacks so next then receives the result in sequence.

Q174. What is async function return value?

An async function always returns a Promise. If it returns a normal value, JavaScript wraps it in a resolved Promise.

Q175. What is try/catch limitation with promises?

try/catch catches errors with await inside async functions. Without await, you need .catch or return the promise properly.

Q176. What is race condition?

Race condition happens when two asynchronous operations complete in unexpected order and the older result may overwrite the newer result. It is common in search APIs.

Q177. How to avoid race condition in search?

Use AbortController to cancel previous requests, or track latest request id and ignore older responses.

Q178. What is currying practical use?

Currying helps create reusable functions with preset values, like createLogger('ERROR') or multiplyBy(2).

Q179. What is partial application?

Partial application means fixing some arguments of a function and returning a new function expecting the remaining arguments.

Q180. What is setTimeout delay guarantee?

setTimeout delay is minimum waiting time, not exact time. Callback runs after stack is empty and after queued tasks before it are processed.

Q181. What is requestAnimationFrame?

requestAnimationFrame schedules visual updates before the next browser repaint. It is better for animations than setInterval.

Q182. What is web worker?

Web Worker runs JavaScript in a background thread, useful for heavy computations without blocking UI. It cannot directly access DOM.

Q183. What is service worker?

Service Worker is a background script used for caching, offline support, push notifications, and intercepting network requests in PWAs.

Q184. What is XSS?

XSS means Cross-Site Scripting. It happens when malicious scripts run in a user's browser. Avoid inserting untrusted HTML and sanitize inputs.

Q185. Why is innerHTML risky?

innerHTML parses HTML. If untrusted data is inserted, attackers may inject scripts or malicious markup. Prefer textContent for user-generated text.

Q186. What is CSRF?

CSRF tricks a logged-in user's browser into making unwanted requests. SameSite cookies and CSRF tokens help reduce this risk.

Q187. What is tree shaking?

Tree shaking removes unused code from final bundle. It works best with ES modules and modern bundlers.

Q188. What is lazy loading?

Lazy loading means loading resources only when needed, such as images or components. It improves initial load performance.

Q189. What is code splitting?

Code splitting divides bundle into smaller chunks so the browser loads only required code initially.

Q190. What is minification?

Minification reduces file size by removing spaces, comments, and shortening names. It improves loading speed.

```
// Function composition
const addOne = x => x + 1;
const double = x => x * 2;
const compose = (f, g) => x => f(g(x));
const doubleThenAddOne = compose(addOne, double);
console.log(doubleThenAddOne(5)); // 11

// Object.freeze is shallow
const user = Object.freeze({ name: "Alex", address: { city: "City" } });
user.address.city = "Delhi";
console.log(user.address.city); // Delhi
```

```

// Avoid older search response overwriting newer response
let latestRequestId = 0;
async function searchUsers(query) {
  const requestId = ++latestRequestId;
  const res = await fetch(`/api/users?q=${query}`);
  const data = await res.json();

  if (requestId !== latestRequestId) return; // ignore old response
  renderUsers(data);
}

// AbortController example
let controller;
async function searchApi(query) {
  if (controller) controller.abort();
  controller = new AbortController();

  try {
    const res = await fetch(`/api/search?q=${query}`, {
      signal: controller.signal
    });
    const data = await res.json();
    console.log(data);
  } catch (err) {
    if (err.name !== "AbortError") console.error(err);
  }
}

```

18. More Tricky Output Practice

Q191. Output?

Async function returns Promise, so direct console gives Promise object.

```
async function test() { return 10; }
console.log(test());
// Output: Promise { 10 } or Promise fulfilled with 10
```

Q192. Output?

await waits for resolved value inside async function.

```
async function test() {
  const x = await Promise.resolve(5);
  console.log(x);
}
test();
// Output: 5
```

Q193. Output?

Object property keys are converted to strings when using normal object.

```
const a = {};
const b = { key: 'b' };
const c = { key: 'c' };
a[b] = 123;
a[c] = 456;
console.log(a[b]);
// Output: 456
```

Q194. Output?

Map can use objects as real keys, so no string conversion conflict.

```
const map = new Map();
const b = { key: 'b' };
const c = { key: 'c' };
map.set(b, 123);
map.set(c, 456);
console.log(map.get(b));
// Output: 123
```

Q195. Output?

Default parameter works when argument is undefined, not when null is passed.

```
function greet(name = 'Guest') { console.log(name); }
greet(undefined);
greet(null);
// Output: Guest, null
```

Q196. Output?

String comparison is lexicographical when both operands are strings.

```
console.log('10' > '2');  
console.log(10 > '2');  
// Output: false, true
```

Q197. Output?

Array length changes when assigning a far index.

```
const arr = [1, 2];  
arr[5] = 10;  
console.log(arr.length);  
// Output: 6
```

Q198. Output?

parseInt stops when it finds invalid character.

```
console.log(parseInt('10px'));  
console.log(Number('10px'));  
// Output: 10, NaN
```

Q199. Output?

Object destructuring with default works only for undefined.

```
const { x = 5 } = { x: undefined };  
const { y = 5 } = { y: null };  
console.log(x, y);  
// Output: 5 null
```

Q200. Output?

Splice mutates original array; slice does not mutate.

```
const arr = [1,2,3,4];  
console.log(arr.slice(1,3));  
console.log(arr);  
console.log(arr.splice(1,2));  
console.log(arr);  
// Output: [2,3], [1,2,3,4], [2,3], [1,4]
```

19. Bonus Coding Round Questions

Q201. Implement chunk array

Break an array into smaller arrays of given size.

```
function chunk(arr, size) {
  const res = [];
  for (let i = 0; i < arr.length; i += size) {
    res.push(arr.slice(i, i + size));
  }
  return res;
}
console.log(chunk([1,2,3,4,5], 2)); // [[1,2],[3,4],[5]]
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q202. Find second largest

Track largest and second largest in one pass.

```
function secondLargest(arr) {
  let first = -Infinity, second = -Infinity;
  for (const n of arr) {
    if (n > first) { second = first; first = n; }
    else if (n > second && n !== first) second = n;
  }
  return second === -Infinity ? null : second;
}
console.log(secondLargest([10, 5, 8, 10, 7])); // 8
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q203. Move zeroes to end

Use two-pointer style. Copy non-zero elements first, then fill zeroes.

```
function moveZeroes(arr) {
  let index = 0;
  for (const n of arr) if (n !== 0) arr[index++] = n;
  while (index < arr.length) arr[index++] = 0;
  return arr;
}
console.log(moveZeroes([0,1,0,3,12])); // [1,3,12,0,0]
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q204. Check if object is empty

Use Object.keys length.

```
function isEmpty(obj) {
  return Object.keys(obj).length === 0;
}
```

```
}
console.log(isEmpty({})); // true
console.log(isEmpty({ a: 1 })); // false
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q205. Convert array to object by id

Use reduce to build lookup object.

```
function arrayToObject(arr) {
  return arr.reduce((acc, item) => {
    acc[item.id] = item;
    return acc;
  }, {});
}
console.log(arrayToObject([{id:1,name:'A'}, {id:2,name:'B'}]));
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q206. Retry promise

Try an async function multiple times before failing.

```
async function retry(fn, attempts) {
  let lastError;
  for (let i = 0; i < attempts; i++) {
    try { return await fn(); }
    catch (err) { lastError = err; }
  }
  throw lastError;
}
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q207. Sleep function

Return a promise that resolves after a delay.

```
function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
async function run() {
  console.log('Start');
  await sleep(1000);
  console.log('After 1 second');
}
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q208. Limit string length

Use slice and add dots if string is longer.


```
function truncate(str, max) {
  if (str.length <= max) return str;
  return str.slice(0, max) + '...';
}
console.log(truncate('JavaScript Interview', 10));
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q209. Find intersection of two arrays

Use Set for faster lookup.

```
function intersection(a, b) {
  const setB = new Set(b);
  return [...new Set(a)].filter(x => setB.has(x));
}
console.log(intersection([1,2,2,3], [2,3,4])); // [2,3]
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q210. Find union of two arrays

Use Set with spread.

```
function union(a, b) {
  return [...new Set([...a, ...b])];
}
console.log(union([1,2], [2,3])); // [1,2,3]
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q211. Count vowels

Check each character against a Set of vowels.

```
function countVowels(str) {
  const vowels = new Set(['a','e','i','o','u']);
  let count = 0;
  for (const ch of str.toLowerCase()) {
    if (vowels.has(ch)) count++;
  }
  return count;
}
console.log(countVowels('JavaScript')); // 3
```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q212. Sum nested array

Use recursion to add numbers inside nested arrays.

```
function sumNested(arr) {
  let sum = 0;
```

```

    for (const item of arr) {
      if (Array.isArray(item)) sum += sumNested(item);
      else sum += item;
    }
    return sum;
  }
  console.log(sumNested([1,[2,[3]],4])); // 10

```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q213. Convert query string to object

Split by & and =, then decode values.

```

function parseQuery(qs) {
  return qs.replace(/\^?/, '').split('&').reduce((acc, pair) => {
    const [key, value] = pair.split('=');
    acc[decodeURIComponent(key)] = decodeURIComponent(value || '');
    return acc;
  }, {});
}
console.log(parseQuery('?name=Alex    role=dev'));

```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q214. Build query string

Use Object.entries and encodeURIComponent.

```

function buildQuery(obj) {
  return Object.entries(obj)
    .map(([k, v]) => `${encodeURIComponent(k)}=${encodeURIComponent(v)}`)
    .join('&');
}
console.log(buildQuery({ name: 'Alex    , role: 'dev' }));

```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.

Q215. Implement basic cache

Use closure with Map to store results.

```

function cacheFunction(fn) {
  const cache = new Map();
  return function(arg) {
    if (cache.has(arg)) return cache.get(arg);
    const result = fn(arg);
    cache.set(arg, result);
    return result;
  };
}

```

Interview tip: Practice dry run after writing the code. Dry run shows interviewer that your logic is correct.